INF201 - Lecture 9

2nd Nov 2026

Topics

- Review of introduction to object-oriented programming
- More on object-oriented programming
  - Composition and inheritance
  - Public, protected and private attributes
  - More special methods

Quick review of object-oriented programming concepts

- The goal of object-orientation is to bundle together data and methods
- Python is an object-oriented language - everything is an object
- Objects are instances of a class
- A class is a template for creating objects
- The class describes attributes, methods and properties

In [1]:
```python
class ClassName:  # create the class ClassName
    """This is a class"""  # Use doc-strings

    CLASS_CONSTANT: int = 100  # Constant for all instances of class (and subcla

    def __init__(self, required_param: str, default_param: int = 10) -> None:
        """Initialise instance of ClassName"""  # Doc-strings and type hints, as
        self.attribute1 = required_param
        self.attribute2 = default_param

    def return_class_constant(self) -> int:
        """Return class constant"""
        return self.CLASS_CONSTANT

    def add_numbers(self) -> int:
        """Return sum of class constant and default param"""
        return self.CLASS_CONSTANT + self.attribute2

    def class_text(self) -> str:
        """Returns string with class info"""
        return f"Class: ClassName. attribute1: {self.attribute1}"
```

In [3]:
```python
obj = ClassName("name")
obj.class_text()
```

Out[3]:
```
'Class: ClassName. attribute1: name'
```

In [4]:
```python
ClassName.CLASS_CONSTANT
```

Out[4]: 100

Composition and inheritance

- These both indicate how two objects relate to each other.
- Compsition: Object a *has* an object b.
  - Composition represents a "has-a" relationship between objects. (from here)
- Inheritance: Class C1 *is* a (subclass of) class C .
  - Inheritance represents an "is-a" relationship between classes. (from here)
- Let's create a couple of classes to demontrate these two concepts.

In [6]:
```python
class Engine:
    """Represents an engine."""

    def __init__(self, horsepower: int = 150) -> None:
        """Create engine with given power"""
        self.horsepower = horsepower

    def start(self) -> str:
        """Create string indicating starting the engine"""
        return f"Engine rumbles to life with {self.horsepower} HP!"
```

In [7]:
```python
class Vehicle:
    """Base vehicle that delegates engine behavior."""

    def __init__(self, brand: str, engine: Engine) -> None:
        """Create a vehicle of specified brand with engine"""
        self.brand = brand
        self.engine = engine  # COMPOSITION: Vehicle has an Engine

    def start_journey(self) -> list[str]:
        """Create list of strings indicating the vehicle has started"""
        engine_msg = self.engine.start()  # Delegate action to the engine compon
        return [engine_msg, f"{self.brand} is on the road."]
```

In [8]:
```python
e1 = Engine(horsepower=200)
v1 = Vehicle("Ford", e1)
v1.start_journey()
```

Out[8]: ['Engine rumbles to life with 200 HP!', 'Ford is on the road.']

- Here we have an example of **composition**: The Vehicle has an Engine.
  - In `Vehicle.start_journey()` the `Vehicle` class delegates `self.engine.start()` to the `Engine` class
  - If we implement changes to the `Engine` class (like including a fuel level check) does not require us to make any changes in `Vehicle`

In [9]:
```python
class Truck(Vehicle):
    """Specialized vehicle using inheritance."""

    def __init__(
        self, brand: str, engine: Engine, max_payload_tons: float = 5.0
    ) -> None:
```

```
        # Pass base attributes up to parent constructor
        super().__init__(brand=brand, engine=engine)
        self.max_payload_tons = max_payload_tons

    def start_journey(self) -> list[str]:
        # Extend parent behavior
        logs = [f"Safety check passed for {self.max_payload_tons}t payload."]
        logs.extend(super().start_journey())
        return logs
```

- This is **inheritance**: `Truck` is a subclass of `Vehicle`
  - `Truck` re-uses the attributes `brand` and `engine` from Vehicle
  - Any attribute or method implemented for `Vehicle` will also be available to `Truck`
  - The sub-class can overwrite or extend class methods if needed, as done here for `start_journey()`

Feature	Inheritance	Composition
Relationship	Is-A	Has-A
Reuse mechanism	Inherits class behavior	Uses another object
Dependency	Child depends on parent hierarchy	Container depends on contained object
Behavior change	Overriding or extending inherited methods	Delegation or replacing contained objects
Class hierarchy	Creates hierarchy	Avoids hierarchy
Flexibility	Less flexible for changing relationships	More flexible through interchangeable objects
Best suited for	Closely related class types	Combining independent responsibilities

Table from here

Inheritance syntax

1. Parent class is included in subclass definition

In [10]:
```
class ClassName:  # create the class ClassName
    """This is a class"""  # Use doc-strings

    CLASS_CONSTANT: int = 100  # Constant for all instances of class (and subcla

    def __init__(self, required_param: str, default_param: int = 10) -> None:
        """Initialise instance of ClassName"""  # Doc-strings and type hints, as
        self.attribute1 = required_param
        self.attribute2 = default_param

    def return_class_constant(self) -> int:
        """Returns class constant"""
```

```
        return self.CLASS_CONSTANT

    def add_numbers(self) -> int:
        """Return sum of class constant and default param"""
        return self.CLASS_CONSTANT + self.attribute2

    def class_text(self) -> str:
        """Returns string with class info"""
        return f"Class: ClassName. attribute1: {self.attribute1}"


class SubClass(ClassName):
    """Subclass of ClassName"""
    pass  # Placeholder until definition is written
```

In [11]:
```
obj_sc_1 = SubClass("name")  # Will use __init__ from parent class
obj_sc_1.attribute1
```

Out[11]:
```
'name'
```

2. If the subclass has it's own `__init__`, it can call the parent initialisation via `super().__init__()`
   - `super()` is a way for the subclass to access the parent class without hardcoding the name of the class
   - Will also work with more complex inheritance hierarchies

In [ ]:
```
class SubClass(ClassName):
    """Subclass of ClassName"""

    def __init__(self, required_param: str, default_param: int = 10, sub_class_p
        """Initialise sub-class object"""
        super().__init__(required_param=required_param, default_param=default_pa
        # ClassName.__init__(required_param=required_param, default_param=defaul
        self.sub_class_attribute = sub_class_param
```

In [13]:
```
obj_sc_2 = SubClass("name", sub_class_param=True)
obj_sc_2.sub_class_attribute
```

Out[13]:
```
True
```

In [14]:
```
obj_sc_2.return_class_constant()  # Inherited from ClassName
```

Out[14]:
```
100
```

3. A subclass can override a parent class method by creating a method of the same name.

In [15]:
```
class SubClass(ClassName):
    """Subclass of ClassName"""

    def __init__(self, required_param: str, default_param: int = 10, sub_class_p
        """Initialise sub-class object"""
        super().__init__(required_param=required_param, default_param=default_pa
        self.sub_class_attribute = sub_class_param
```

```
    def add_numbers(self):
        """Add 10 to default parameter"""
        return self.attribute2 + 10
```

4. A subclass can extend a parent class method by calling `super().method_name()`.

In [16]:
```
class SubClass(ClassName):
    """Subclass of ClassName"""

    def __init__(self, required_param: str, default_param: int = 10, sub_class_p
        """Initialise sub-class object"""
        super().__init__(required_param=required_param, default_param=default_pa
        self.sub_class_attribute = sub_class_param

    def add_numbers(self):
        """Add 10 to default parameter"""
        return self.attribute2 + 10

    def class_text(self) -> str:
        """Returns string with class info"""
        txt_sc = "Subclass: SubClass. "
        txt_from_class = super().class_text()
        return txt_sc + txt_from_class
```

In [17]:
```
obj_sc_3 = SubClass("name")
obj_sc_3.class_text()
```

Out[17]:
```
'Subclass: SubClass. Class: ClassName. attribute1: name'
```

A few things to note here:

- Why does `super()` not take `self` as an argument?
  - Python does this for you!
    `super() -> super(__class__, self)`
- Why do I return strings rather than printing directly?
  - This separates logic from presentation (I/O)
  - When the class/method is used, we can chose whether we want to print the output or not
  - We can also collect the output strings from several methods or runs into a log

```
def some_method(self, verbose: bool = False) -> list[str]:
"""Does something that produces a log."""
all_logs: list[str] = []

# Something happens
first_logs = self.first_method()
all_logs.extend(first_logs)

# Something else happens
sencond_logs = self.second_method()
all_logs.extend(second_logs)

# Verbose controls console output
if verbose:
    for log in all_logs:
```

```
                print(log)

        return all_logs
```

Practice example:

Let's create a Pet class, with Dog and Cat subclasses, where each pet can have a Toy (object of Toy class)

In [21]:
```
class Toy:
    """Represents a toy"""

    def __init__(self, name: str) -> None:
        """Create a toy with a name"""
        self.name = name


class Pet:
    """Represents any pet"""

    def __init__(self, name: str, toy: Toy | None = None) -> None:
        """Initialise pet"""
        self.name = name
        self.toy = toy

    def talk(self) -> str:
        """Let pet talk"""
        return "I can't talk"


class Dog(Pet):
    """Represents a dog"""

    def talk(self) -> str:
        """Let dog talk"""
        return "BARK!"


class Cat(Pet):
    """Represents a cat"""

    def talk(self) -> str:
        """Let cat talk"""
        return "Meow?"


ball = Toy("ball")
tassa = Cat("Tassa", ball)
tassa.talk()
```

Out[21]:
```
'Meow?'
```

Public, protected and private attributes

- We talked a bit about public and protected attributes last week
- Public attributes

    - Syntax: No leading underscores (`self.name`).
    - Behavior: Accessible and modifiable from anywhere—inside the class, by subclasses, and from outside instance calls.
    - When to use: For attributes and methods that are intended to be part of the class's public interface.
- Protected attributes
    - Syntax: A single leading underscore (`self._name`).
    - Behavior: Serves as a convention telling other developers, "This is internal logic/data—do not touch it directly outside the class unless you are writing a subclass." Python does *not* strictly enforce this at runtime. The code will still run if accessed externally, but tools like VS Code/Pylance will flag warnings.
- Private attributes
    - Syntax: Double leading underscores without trailing underscores (`self.__name`).
    - Behavior: Triggers name mangling. Python automatically rewrites the attribute name under the hood by prepending `_ClassName` (such as `_ClassName__name`). This prevents accidental overwriting or external access using the original name.
    - When to use: To avoid name collisions in subclassing, or to make accidental direct access noticeably harder.

Access Modifier	Python Syntax	Access from Outside Class	Access from Subclass	Enforcement Level
Public	`self.name`	Allowed	Allowed	None (Open access)
Protected	`self._name`	Discouraged	Allowed	Convention only (Linter warnings)
Private	`self.__name`	Blocked via standard name	Blocked via standard name	Name mangling (`_ClassName__name`)

- Why do we not use private attributes more?
    - It breaks inheritance because the attribute name is strictly bound to the class where it was defined

```
class Character:
    """A game character"""

    def __init__(self, hp: int) -> None:
        """Initialise character with HP"""
        self.__hp = hp  # Mangled to _Character__hp


class Warrior(Character):
    """A warrior character"""

    def heal(self, amount: int) -> None:
        """Heal for amount HP"""
        self.__hp += amount  # Fails with AttributeError:
'Warrior' object has no attribute '_Warrior__hp'
```

- It provides a false sense of security

```
ch1 = Character(hp=25)
print(ch1.__hp)  # Fails with AttributeError: 'Character' object has no attribute '__hp'
print(ch1._Character__hp)  # Can still be accessed using the mangled name
ch1._Character__hp = -999  # Can still be changed using the mangled name
```

- Code is harder to read, test and debug
- Python's community standards expect you to respect the protected attribute (`_name`), making the need for a private attribute (`__name`)
- Use a single underscore (_) by default whenever an attribute or method is internal. Only use double underscores (__) if you are building deep inheritance structures where name collisions with unknown subclasses become a real risk.

In [22]:
```
class Character:
    """A game character"""

    def __init__(self, hp: int) -> None:
        """Initialise character with HP"""
        self.__hp = hp  # Mangled to _Character__hp


ch1 = Character(hp=25)
print(f"Using mangled name: {ch1._Character__hp}")
print(f"Using regular name: {ch1.__hp}")
```

```
Using mangled name: 25
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[22], line 11
      7
      8
      9 ch1 = Character(hp=25)
     10 print(f"Using mangled name: {ch1._Character__hp}")
---> 11 print(f"Using regular name: {ch1.__hp}")

AttributeError: 'Character' object has no attribute '__hp'
```

Special methods

- These methods allow custom objects to integrate with Python built-in functions (`len()` , `str()` , `print()`),
  operators (`+` , `==` , `in`), and control flow (`for` , `with`)
- Dunder methods, magic methods, ...
- List of all special methods
- Names with double underscores, like `__init__`
- Last week we looked at
  - `__init__` for creating (initalising) an object
  - `__repr__` for object representation (for displaying the object)
  - `__str__` for object string representation (for printing the object)

Methods for sequences and containers

- `__len__`
  - Determines what `len(class_object)` should return
  - If a class does not have `__len__` defined, `len(class_object)` will raise a `TypeError`
  - Must be $>= 0$
  - If `__bool__` (for determining truthiness) is not defined for the class, Python uses `__len__` (0 is falsy, other values are truthy)

In [23]:
```
text1 = ""
if text1:
    print("Truthy")
else:
    print("Falsy")
```

```
Falsy
```

In [24]:
```
text2 = "text"
if text2:
    print("Truthy")
else:
    print("Falsy")
```

```
Truthy
```

In [25]:
```
print(dir(str))
```

```
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__
eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__getnewargs_
_', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__ite
r__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__',
'__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__',
'__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center',
'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'ind
ex', 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'is
lower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'lju
st', 'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix', 'removesuffix',
'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split',
'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper',
'zfill']
```

In [26]:
```
str.__bool__?
```

```
Object `str.__bool__` not found.
```

- `__getitem__`
  - Allows index and key look-up, such as `class_object[0]` or `class_object["key_word"]`
  - Allows slicing, such as `class_object[0:2]`
- `__contains__`
  - Does the object contain the specified item? `item in class_object`

In [27]:
```
# See file dataset_with_type_hints.py for version with full type hints and doc-s
class Dataset:
```

```
    def __init__(self, records: list) -> None:
        self._records = records

    def __len__(self) -> int:
        return len(self._records)

    def __getitem__(self, index):
        # Enables d[0], d[1:3], and even iteration for free!
        return self._records[index]

    def __contains__(self, item) -> bool:
        # Enables 'item in dataset'
        return item in self._records
```

In [28]:
```
d = Dataset(["a", "b", "c"])
d
```

Out[28]:
```
<__main__.Dataset at 0x2070f0f6a50>
```

In [29]:
```
len(d)
```

Out[29]:
```
3
```

In [30]:
```
d[0:2]
```

Out[30]:
```
['a', 'b']
```

In [31]:
```
"c" in d
```

Out[31]:
```
True
```

In [32]:
```
for element in d:
    print(element)
```

```
a
b
c
```

Methods for comparisons

- `__eq__`
  - Customises the `==` operator
  - What does it mean for two objects to be considered equal?
  - If not implemented, `==` will default to checking object identity (memory address)
  - `True`, `False` or `NotImplemented`
- `__lt__`
  - Customises the `<` operator
  - How do we check if this object is less than another object?
- `__le__`, `__gt__`, and `__ge__`
  - Customises `<=`, `>` and `>=`
  - All about comparison of objects

- By using `total_ordering` from `functools`, you can define `__eq__` and one of the other comparisons in the class, and the rest will be implemented by total_ordering
  - `@total_ordering` is another example of a decorator, like `@property` we used last week

In [ ]:
```python
from functools import total_ordering


@total_ordering
class Vector:

    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def __eq__(self, other) -> bool:
        if not isinstance(other, Vector):
            return NotImplemented
        return self.x == other.x and self.y == other.y

    def __lt__(self, other) -> bool:
        if not isinstance(other, Vector):
            return NotImplemented
        return (self.x**2 + self.y**2) < (other.x**2 + other.y**2)
```

Methods for context management

- `__enter__` and `__exit__`
  - For working with contexts (`with`)
  - Determines what happens when we enter and exit the context
  - More about creating context managers here

In [33]:
```python
import time


class Timer:

    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.elapsed = time.perf_counter() - self.start
        print(f"Execution took: {self.elapsed:.4f} seconds")
        return False  # Do not suppress exceptions
```

In [34]:
```python
with Timer():
    # Code block being timed
    sum(i**2 for i in range(10_000_000))
```

```
Execution took: 0.5701 seconds
```

Method for making an object callable

- Allows you to call the object as if it was a function

In [ ]:
```python
class Counter:

    def __init__(self):
        self.count = 0

    def __call__(self):
        self.count += 1
        return self.count


# Create a counter object
my_counter = Counter()
# Call the object multiple times
print(my_counter())  # Output: 1
print(my_counter())  # Output: 2
print(my_counter())  # Output: 3
```

```
1
2
3
```

Why do objects have methods that we have not implemented?

- Every class in Python implicitly inherits from the `object` class

```python
class NewClass():  # What we write
    pass

class NewClass(object):  # What happens behind the scene
    pass
```

- We used this last week when we looked at how everything is an object

```python
print(isinstance(42, object))  # True
print(isinstance("hello", object))  # True
print(isinstance([1, 2, 3], object))  # True
print(isinstance(NewClass, object))  # True (even classes are objects!)
```